

Beyond: Better-than-Full-Precision KV Caches via Learnable Non-Uniform Quantization as Implicit Regularization

Yuhao Xie Mingjie Lin
University of Central Florida
4000 Central Florida Blvd.
Orlando, FL 32816, USA
{yu805059, milin}@ucf.edu
Correspondence: milin@ucf.edu

Abstract

Key-value (KV) caching accelerates autoregressive decoding but makes long-context serving memory-intensive. Most KV quantizers minimize local reconstruction distortion, although this proxy ignores how perturbations propagate through downstream computation. We introduce **Beyond**, a loss-aware non-uniform quantizer that learns representation levels and decision thresholds from next-token negative log-likelihood while freezing the backbone. Training and inference use the same hard quantizer, with straight-through and boundary-localized pseudo-gradients updating only its parameters. With 4-bit codes and group size 32, **Beyond** uses 5.0 effective bits per KV scalar, providing $3.2\times$ compression relative to a 16-bit cache. Across three instruction-tuned LLMs, it lowers held-out Pile perplexity below the full-precision baseline by 2.2–7.0% and attains the highest LongBench Avg21 among full-precision Native, KV-FP8, and TurboQuant for each model. On an NVIDIA B300, its fused packed-cache kernel achieves a peak $1.50\times$ speedup over FlashAttention-4, while the resulting vLLM path improves end-to-end throughput by up to $1.17\times$. Our code is available at <https://github.com/wsxhjnb1/Beyond-KV-Cache>.

1 Introduction

In modern decoder-only LLM families such as Qwen3 and Llama 3 (Yang et al., 2025; Grattafiori et al., 2024), key-value (KV) caching is the standard mechanism for efficient autoregressive decoding: past keys and values are stored so each step computes only the new token’s states. At long context, this creates a KV-memory bottleneck because cache size scales linearly with sequence length.

KV-memory reduction spans low-rank projection (Chang et al., 2025; Saxena et al., 2024), token eviction/merging (Zhang et al., 2023, 2024b), and quantization (Liu et al., 2024c; Hooper et al., 2024).

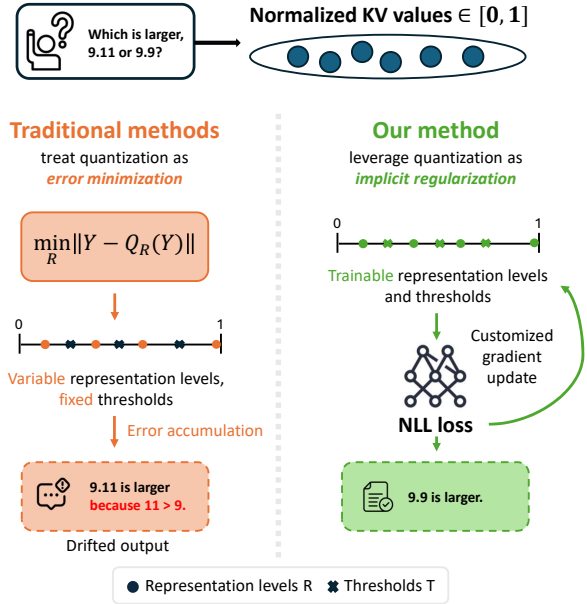


Figure 1: **From reconstruction-error minimization to loss-aware KV-cache quantization.** Conventional KV quantization fits local reconstruction objectives and then freezes the quantizer, while **Beyond** keeps the deployment-time hard quantizer but optimizes representation levels R and decision thresholds T against the language-model loss.

Low-rank methods reduce the representational dimension of each cached state, and token-level methods can discard, merge, or blur individual context tokens, while quantization preserves the full token history and original KV dimensionality.

Most KV quantizers optimize a local distortion surrogate. Uniform schemes are efficient, but fixed bins can poorly match the channel- and token-dependent statistics of KV states (Liu et al., 2024c; Su et al., 2025); non-uniform, decomposition-based, and vector-compression methods improve this fit using calibration data or online distortion objectives (Hooper et al., 2024; Liu et al., 2024b; Zandieh et al., 2026). Nevertheless, aggressive

KV compression remains lossy: perturbing cached keys and values can distort attention, increase perplexity, and reduce retrieval or downstream-task accuracy, with the risk becoming more pronounced at very low bit widths (He et al., 2024; Kang et al., 2024; Su et al., 2025). **Deployers must therefore trade memory footprint and serving efficiency against model quality, narrowing the practical value of KV compression when even small accuracy losses are unacceptable.** Local reconstruction error does not resolve this trade-off because it is only a proxy for generation loss: KV perturbations propagate through attention and subsequent computation before affecting token probabilities (Fig. 1). Thus, a quantizer that better reconstructs KV tensors locally need not better preserve end-task behavior.

Our token-level diagnostics show that even uniform low-bit KV quantization can lower perplexity on some positions, suggesting that discretization is not purely destructive. This motivates treating KV quantization as a constrained adaptation problem: *can a compressed, hard-forward KV cache be trained to reduce language-model loss?* We propose **Beyond**, which learns non-uniform levels and thresholds with the backbone frozen. The forward pass is the exact hard quantizer used at inference, and the backward pass uses STE-style pseudo-gradients for representation levels and thresholds, turning quantization from a fixed calibration artifact into a lightweight loss-aware adaptation.

Our contributions are:

- **Loss-aware quantization principle.** We show that reconstruction error is a degenerate local objective: downstream loss also depends on perturbation mean and anisotropic curvature.
- **Deployment-exact learning.** **Beyond** learns non-uniform levels and thresholds with a hard forward pass that matches inference and STE-style backward updates.
- **Custom decode kernel.** We implement a customized FlashAttention-style decoding kernel that consumes the learned packed KV cache directly for compressed-cache inference.

2 Preliminaries and Related Works

This section reviews the background needed for KV-cache quantization and related memory-reduction techniques.



Figure 2: **Llama-3.1-8B-Instruct Six-Benchmark Comparison.** Scores on the six benchmark tasks. Higher is better.

2.1 Preliminaries

We first review KV caching and then define standard scalar quantization for KV states.

KV cache for multi-head attention. In decoder-only LLMs, e.g., LLaMA (Touvron et al., 2023), key/value (KV) caching accelerates autoregressive inference by storing the keys and values produced for previous tokens. Given a prefix $X = [x_1, \dots, x_t]$, for each layer and attention head i , the cache before processing x_t contains $\mathbf{K}_i^{t-1}$ and $\mathbf{V}_i^{t-1}$ for $x_{1:t-1}$.

To process token x_t , the model computes the query, key, and value vectors from its hidden state $\mathbf{h}_t$ (we omit the layer index and positional encodings for notational simplicity):

$$\mathbf{q}_i^t = \mathbf{h}_t \mathbf{W}_{Q,i}, \mathbf{k}_i^t = \mathbf{h}_t \mathbf{W}_{K,i}, \mathbf{v}_i^t = \mathbf{h}_t \mathbf{W}_{V,i},$$

where $\mathbf{W}_{Q,i}$, $\mathbf{W}_{K,i}$, and $\mathbf{W}_{V,i}$ are the head-specific projection matrices. Positional information is incorporated before attention scores are formed: the original Transformer adds encodings to token embeddings (Vaswani et al., 2017), whereas RoPE rotates query/key states (Su et al., 2024); both are omitted here to keep the notation focused on KV caching. The new key and value are appended to the cache, $\mathbf{K}_i^t = \text{Concat}(\mathbf{K}_i^{t-1}, \mathbf{k}_i^t)$ and $\mathbf{V}_i^t = \text{Concat}(\mathbf{V}_i^{t-1}, \mathbf{v}_i^t)$, after which attention is

$$\text{Attn}(\mathbf{q}_i^t, \mathbf{K}_i^t, \mathbf{V}_i^t) = \text{Softmax}\left(\frac{\mathbf{q}_i^t (\mathbf{K}_i^t)^\top}{\sqrt{d}}\right) \mathbf{V}_i^t,$$

where d is the per-head hidden dimension shared by queries and keys, and the softmax is applied over cached positions. KV caching therefore avoids recomputing keys and values for the full prefix at every decoding step.

The memory footprint of KV cache grows linearly with the sequence length $S = |X|$, as shown in Formula 1:

$$\text{size}(\mathbf{K}, \mathbf{V}) = 2 \times B \times S \times L \times h \times d \times b_{\text{dtype}}, \quad (1)$$

where $\mathbf{K}$ and $\mathbf{V}$ denote the KV cache across h heads, B is the batch size, L is the number of layers, and b_{dtype} is the number of bytes per stored element in memory. While providing critical performance gains, the additional memory footprint of KV cache can quickly saturate the limited device memory and become the most stringent bottleneck for LLM inference.

Hard scalar KV quantization. For a cached scalar z in a quantization group, scalar quantization commonly begins with groupwise affine normalization:

$$x = \text{clip}\left(\frac{z - z_{\min}}{z_{\max} - z_{\min} + \epsilon}, 0, 1\right).$$

The hard b -bit scalar quantizer has $K = 2^b$ bins, representation values $R = \{r_k\}_{k=0}^{K-1}$, and ordered thresholds $T = \{t_k\}_{k=0}^K$ with fixed endpoints $t_0 = 0$ and $t_K = 1$. Let $B_k(T) = [t_k, t_{k+1})$ for $k < K - 1$ and $B_{K-1}(T) = [t_{K-1}, t_K]$. The normalized hard quantizer is

$$Q(x; R, T) = \sum_{k=0}^{K-1} r_k \mathbf{1}\{x \in B_k(T)\}. \quad (2)$$

The corresponding dequantized value is

$$\hat{z} = z_{\min} + (z_{\max} - z_{\min}) Q(x; R, T).$$

Uniform quantization is the special case where the representation values are evenly spaced:

$$r_k = \frac{k}{K-1}, \quad k = 0, \dots, K-1, \quad (3)$$

$$t_k = \frac{r_{k-1} + r_k}{2}, \quad k = 1, \dots, K-1. \quad (4)$$

By contrast, non-uniform quantization allows unequal spacings in R and T . Prior non-uniform KV quantization methods often choose these parameters through clustering (Hooper et al., 2024).

2.2 Related Works

KV-cache quantization. KV-cache quantization reduces memory usage by compressing cached keys and values to lower bit-widths. KIVI (Liu et al., 2024c) uses asymmetric uniform quantization, applying per-channel quantization to keys and per-token quantization to values. KVQuant (Hooper et al., 2024) fits per-layer

non-uniform quantizers with k-means and a Fisher-weighted objective, while CQQuant (Zhang et al., 2024a) jointly quantizes coupled key/value channels to exploit inter-channel dependence at very low bit-widths. NSNQuant (Son et al., 2025) normalizes KV activations toward a standard normal prior and applies low-bit vector quantization with a reusable codebook. GEAR (Kang et al., 2024) instead augments quantized KV states with residual corrections. TurboQuant (Zandieh et al., 2026) applies an online vector-compression scheme based on randomized rotations, scalar quantization of rotated coordinates, and Quantized JL residual correction. These methods mainly optimize reconstruction, distortion, or calibration criteria, whereas **Beyond** optimizes quantizer parameters against the downstream language-model loss.

Other KV-cache reduction strategies. Low-rank methods such as Palu (Chang et al., 2025), Eigen Attention (Saxena et al., 2024), and ASVD’s KV-cache extension (Yuan et al., 2023) reduce KV channel dimensions. Token eviction methods, including Heavy-Hitter Oracle (Zhang et al., 2023), PyramidKV (Cai et al., 2025), SnapKV (Li et al., 2024), and StreamingLLM (Xiao et al., 2024), shrink the cache by discarding tokens, while cache merging or fusion methods such as CaM (Zhang et al., 2024b), Adaptive KV Merging (Wang et al., 2024), MiniCache (Liu et al., 2024a), and ZS-Merge (Liu et al., 2025) combine redundant cache states. These approaches are largely orthogonal to quantization, but they may remove or blur information that later tokens require.

Learnable quantization via STE. Straight-through estimators replace the zero or undefined derivative of discrete operators with surrogate gradients during backpropagation (Bengio et al., 2013). In quantization-aware training, this allows the forward pass to keep a hard quantizer while selected quantizer parameters are learned from task loss. LSQ learns quantizer step sizes for weights and activations (Esser et al., 2020), and PACT learns activation clipping bounds (Choi et al., 2018). DSQ relaxes quantization with differentiable soft assignments that are annealed toward hard decisions (Gong et al., 2019). N2UQ learns non-uniform input thresholds while keeping uniform output levels through a generalized STE for threshold parameters (Liu et al., 2022). In LLM KV-cache activation quantization, however, end-to-end learning that jointly optimizes representation levels and decision thresholds with downstream language-

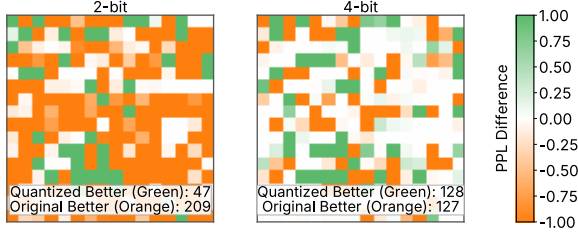


Figure 3: **Token-level PPL differences under uniform KV quantization on a chat-wrapped The Pile sample for Llama-3.1-8B-Instruct, with 2-bit (left) and 4-bit (right).** Heatmaps show $\Delta\text{PPL} = \text{PPL}_{\text{float16}} - \text{PPL}_{\text{quant}}$ over 256 supervised target tokens. Quantization uses group-32 per-token normalization over the head dimension for both K and V. Green indicates lower token PPL after quantization and orange indicates degradation; a non-trivial subset of tokens improves over the full-precision baseline, especially at 4-bit.

model loss remains limited.

3 Methodology

Beyond treats KV-cache quantization as loss-aware hard-forward non-uniform quantization: it learns representation levels R and thresholds T by minimizing next-token NLL under a hard quantized forward pass. At inference, the learned quantizers are used in a real-compression path where optimized CUDA kernels execute attention directly over the packed KV cache, keeping the training and inference quantization semantics matched.

3.1 Observations and Motivations

Figures 3 and 4 provide the Llama-3.1-8B-Instruct (Grattafiori et al., 2024) diagnostics used to motivate this section: token-level PPL changes under low-bit uniform quantization on a The Pile (Gao et al., 2020) sample formatted with the chat-prefix recipe described in Appendix A, and layer-wise KV activation heterogeneity. In Fig. 3, uniform 2-bit quantization lowers token PPL on 47 of 256 scored positions (18.4%), while 4-bit quantization improves 128 of 256 positions (50.0%). Thus the quantization perturbation is signed and token-dependent rather than uniformly harmful. Fig. 4 further shows that min-max normalized K/V activations remain concentrated but shift across layers and differ between keys and values. This agrees with prior observations that KV activations place most mass in a narrow range while a small outlier

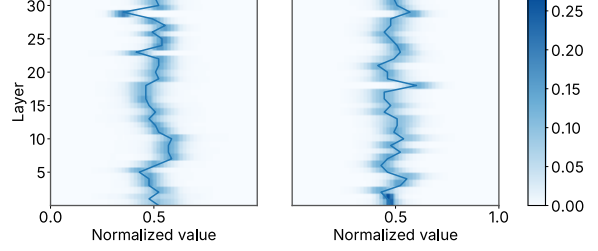


Figure 4: **Per-layer heatmaps of min-max normalized post-RoPE key (Left) and post-projection value (Right) activations in Llama-3.1-8B-Instruct for the same The Pile chat-wrapped sample.** Color indicates empirical density over normalized activation values, and the overlaid curve traces the modal bin for each layer. The concentrated mass, layer-dependent shifts, and K/V differences show why fixed uniform levels can mismatch the activation geometry.

fraction reaches much larger magnitudes (Hooper et al., 2024), and that atypical tokens can disproportionately affect quantization fidelity (Su et al., 2025). The resulting geometry favors layer-aware non-uniform levels over fixed uniform levels; fixed uniform schemes such as KIVI (Liu et al., 2024c) cannot adapt representation density to these shifts.

These observations motivate a loss-aware view of KV quantization. Existing KV-cache compressors fit fixed local surrogates, including sensitivity- or Fisher-weighted fitting, residual correction, normalized codebooks, and online distortion criteria (Hooper et al., 2024; Zhang et al., 2024a; Kang et al., 2024; Son et al., 2025; Zandieh et al., 2026). Downstream NLL can rank these perturbations differently: equally well-calibrated perturbations can align differently with the local loss gradient and curvature. The expansion below formalizes this gap and identifies when it collapses back to reconstruction-style fitting.

Proposition 3.1 (Local loss-aware quantization principle). *Let $\Psi(z)$ be the token loss induced by cached representation z in a frozen downstream computation, and let a quantizer with parameters φ induce perturbation $\delta_\varphi = Q_\varphi(z) - z$. If Ψ has β -Lipschitz Hessian in operator norm in a neighborhood of z , with $g = \nabla\Psi(z)$ and $H = \nabla^2\Psi(z)$, then*

$$\begin{aligned} \mathbb{E}\Psi(z + \delta_\varphi) - \Psi(z) &= \langle g, m_\varphi \rangle \\ &\quad + \frac{1}{2} \text{Tr}(H C_\varphi) + \rho_\varphi, \\ |\rho_\varphi| &\leq \frac{\beta}{6} \mathbb{E}\|\delta_\varphi\|_2^3, \end{aligned}$$

where $m_\varphi = \mathbb{E}[\delta_\varphi]$ and $C_\varphi = \mathbb{E}[\delta_\varphi \delta_\varphi^\top]$. Thus local loss-aware fitting depends on gradient alignment $\langle g, m_\varphi \rangle$ and Hessian-weighted second moment $\text{Tr}(HC_\varphi)$, not only a calibration distortion. Fisher-weighted distortion is recovered only after discarding the first-order term and replacing the possibly indefinite downstream Hessian H with a fixed positive-semidefinite generalized Gauss–Newton/Fisher matrix H_{GNN} ; plain reconstruction additionally requires $H_{\text{GNN}} = \lambda I$ for some $\lambda > 0$.

Proof. See Appendix G.1.

For KV caches, the perturbation enters through attention. A threshold shift that slightly increases local reconstruction error can still reduce negative log-likelihood if it suppresses a competing continuation preferred by the full-precision model. We therefore treat KV quantization as a loss-aware perturbation rather than as pure numerical error, and instantiate the principle through a local attention decomposition (Propositions 3.2–3.3).

Proposition 3.2 (Loss-aware perturbation view of KV quantization). *Under the first-order model in Appendix G.1, let $\Delta o = (J_{\text{sm}}(a)\varepsilon)V + p \Delta V$ denote the linearized attention-output perturbation and define $o_{\text{lin}} = o + \Delta o$. Let $\mu = \mathbb{E}[\Delta o]$. If the downstream loss $\ell(\cdot)$ is L -smooth, then*

$$\mathbb{E}[\ell(o_{\text{lin}})] \leq \ell(o) + \langle \nabla \ell(o), \mu \rangle + \frac{L}{2} \mathbb{E} \|\Delta o\|_2^2,$$

where $a = \frac{1}{\sqrt{d}} q K^\top$, $p = \text{Softmax}(a)$, $o = pV$. If $\langle \nabla \ell(o), \mu \rangle < -\frac{L}{2} \mathbb{E} \|\Delta o\|_2^2$, the quantized linearized output has strictly lower expected loss than the full-precision output.

Proof. See Appendix G.1.

Proposition 3.3 (Competing-path margin suppression). *Let $s_j(o_t) = w_j^\top o_t$ and $\pi_t = \text{Softmax}(W o_t)$. For a competing token $j \neq y_t$, let $\Delta m_t(j)$ be the change in its logit margin over the target token after perturbing o_t by μ_t :*

$$\begin{aligned} \Delta m_t(j) &\triangleq [s_j(o_t + \mu_t) - s_{y_t}(o_t + \mu_t)] \\ &\quad - [s_j(o_t) - s_{y_t}(o_t)] \\ &= \langle w_j - w_{y_t}, \mu_t \rangle. \end{aligned}$$

The cross-entropy first-order term is the softmax-weighted average of these one-token shifts:

$$\langle \nabla_{o_t} \ell_t(o_t), \mu_t \rangle = \sum_{j \neq y_t} \pi_{t,j} \Delta m_t(j).$$

For a competing continuation $c_{1:T}$, use the same teacher-forced states o_t from the target prefixes $y_{<t}$,

and compare the path margins before and after perturbation:

$$\begin{aligned} M(c, y) &= \sum_t (s_{c_t}(o_t) - s_{y_t}(o_t)), \\ M_\mu(c, y) &= \sum_t (s_{c_t}(o_t + \mu_t) - s_{y_t}(o_t + \mu_t)). \end{aligned}$$

Their difference simply selects the one-token shift on the path:

$$\begin{aligned} \Delta M(c, y) &\triangleq M_\mu(c, y) - M(c, y) \\ &= \sum_{t=1}^T \Delta m_t(c_t) = \sum_{t=1}^T \langle w_{c_t} - w_{y_t}, \mu_t \rangle. \end{aligned}$$

Thus $\Delta M(c, y) < 0$ means the perturbation suppresses path c ; cross-entropy uses the same shifts, averaged over all competing tokens.

Proof. See Appendix G.1.

Together, the propositions give an operational criterion: choose representation levels and thresholds by their effect on downstream NLL, rather than by a separate calibration surrogate.

Accordingly, we learn the quantization levels R and thresholds T by minimizing the language-model negative log-likelihood under KV cache quantization:

$$\begin{aligned} (R^*, T^*) &= \arg \min_{R, T} \sum_{X \in \mathcal{X}} \sum_{t=1}^{|X|-1} \\ &\quad - \log p_\theta(x_{t+1} \mid x_{\leq t}; Q_{R, T}). \end{aligned} \quad (5)$$

where $\mathcal{X}$ is the training sequence stream, θ denotes the base model parameters, and $p_\theta(\cdot \mid x_{\leq t}; Q_{R, T})$ is the next-token distribution when the cached K/V states produced while processing $x_{\leq t}$ are quantized by the b -bit scalar quantizer $Q_{R, T}$. The quantizer is parameterized by representation values R and ordered decision thresholds T .

3.2 Hard-Forward STE for Learnable Non-Uniform KV Quantization

To enable gradient-based updates of both representation levels and decision thresholds, **Beyond** uses a deployment-exact hard forward pass together with a standard STE input rule and explicit hard-forward pseudo-gradients for the learnable quantizer parameters.

Hard forward quantization. For each normalized K/V group, the forward pass simply applies the scalar quantizer in Eq. (2): $\hat{x} = Q(x; R, T)$.

This is the same hard operator used at deployment. Because the endpoints remain fixed at $t_0 = 0$ and $t_K = 1$, optimization updates only the levels R and the interior thresholds $\{t_k\}_{k=1}^{K-1}$.

Non-differentiability and standard STE input backward pass. The hard quantizer in Eq. (2) is piecewise constant in x and changes assignments only when a threshold crosses a sample. Consequently, the classical derivative with respect to x is zero almost everywhere, and the classical derivative with respect to a threshold is either zero away from assignment changes or undefined at assignment changes, making it unsuitable for gradient-based optimization. Quantization-aware training therefore often uses a straight-through estimator (STE), which keeps the hard forward value while substituting a surrogate derivative in the backward pass (Bengio et al., 2013; Hubara et al., 2018). We use the stop-gradient form

$$\tilde{x} = x + \text{sg}(Q(x; R, T) - x), \quad (6)$$

where $\text{sg}(\cdot)$ denotes stop-gradient. The forward value is exactly $\hat{x} = Q(x; R, T)$, matching deployment, while the backward pass treats the quantizer as the identity map:

$$\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial \hat{x}}. \quad (7)$$

This standard STE input rule introduces no auxiliary gates, temperatures, or annealing schedules; training and inference use the same hard bin assignment.

Updates for quantization levels $R = \{r_k\}_{k=0}^{K-1}$. Under the hard forward quantizer (2), each level only affects samples assigned to bin k , i.e., $\partial \hat{x} / \partial r_k = \mathbb{I}[x \in \text{bin}_k]$. Thus, for a minibatch $\mathcal{B}$,

$$\frac{\partial L}{\partial r_k} = \sum_{x \in \mathcal{B}} \mathbb{I}[x \in \text{bin}_k] \frac{\partial L}{\partial \hat{x}}.$$

Updates for thresholds $T = \{t_k\}_{k=1}^{K-1}$ with half-wave rectification. An interior threshold t_k only changes assignments for samples near the boundary between adjacent reconstruction levels r_{k-1} and r_k . Locally, the K -level quantizer can be approximated by a 2-level switch:

$$\hat{x} = r_{k-1} + \Delta r_k H(x - t_k), \quad \Delta r_k \triangleq r_k - r_{k-1},$$

where $H(\cdot)$ is the Heaviside step function. Since $H'(u) = \delta(u)$ in the distributional sense,

$$\frac{\partial \hat{x}}{\partial t_k} = -\Delta r_k \delta(x - t_k).$$

The impulse $\delta(x - t_k)$ cannot be used directly in backpropagation, so we approximate its boundary effect by aggregating gradients in a narrow window of width ϵ_{bw} around t_k . For a minibatch $\mathcal{B}$, define

$$\mathcal{W}_L = \{x \in \mathcal{B} : t_k - \epsilon_{\text{bw}} \leq x < t_k\},$$

$$\mathcal{W}_R = \{x \in \mathcal{B} : t_k \leq x \leq t_k + \epsilon_{\text{bw}}\}.$$

and the corresponding aggregated boundary signals

$$G_L = -\Delta r_k \sum_{x \in \mathcal{W}_L} \frac{\partial L}{\partial \hat{x}},$$

$$G_R = -\Delta r_k \sum_{x \in \mathcal{W}_R} \frac{\partial L}{\partial \hat{x}}.$$

The two sides represent directional candidates rather than additive effects. A small left shift ($t_k \downarrow$) moves samples in $\mathcal{W}_L$ from r_{k-1} to r_k and is loss-decreasing iff $G_L > 0$; a small right shift ($t_k \uparrow$) moves samples in $\mathcal{W}_R$ from r_k to r_{k-1} and is loss-decreasing iff $G_R < 0$. We therefore suppress the opposite, loss-increasing directions with half-wave rectification:

$$\hat{\nabla}_{t_k} L = [G_L]_+ + [G_R]_-, \quad (8)$$

where $[u]_+ = \max(u, 0)$ and $[u]_- = \min(u, 0)$. Under gradient descent, $[G_L]_+$ pushes t_k left and $[G_R]_-$ pushes it right, while noisy or disagreeing boundary evidence is discarded. After each optimizer step, we project $\{t_k\}_{k=1}^{K-1}$ back to the ordered interval $t_1 < \dots < t_{K-1}$.

Optimizer and theoretical scope. We update only the quantizer parameters $\varphi \triangleq (R, T)$ with Adam (Kingma and Ba, 2015); the backbone weights remain frozen. The level and threshold updates are hard-forward pseudo-gradients driven by downstream NLL, not by a separate Euclidean reconstruction objective. Appendix G.3 gives a conditional projected-AMSGrad stationarity check: if the pseudo-gradient is treated as a bounded-mismatch stochastic gradient oracle for a smooth reference objective, then the standard projected-AMSGrad bound applies.

3.3 Fused Decode Kernel

To realize the HBM I/O benefit of KV-cache quantization, decode attention must consume the packed representation directly rather than materializing dense K/V in HBM. Related low-bit systems likewise fuse KV dequantization into attention computation (Lin et al., 2025; Yao et al., 2026). **Beyond** keeps prefill on the standard dense-attention path: its writer stores persistent 4-bit K/V codes and 16-bit groupwise affine statistics, while prefill attention consumes model-dtype quantize–dequantize (QDQ) K/V. For $a \in \{K, V\}$, omitting layer, token, head, and group indices,

$$\mathcal{D}_a(c) = m_a + s_a R_a[c],$$

where R_K and R_V are independent 16-entry reconstruction tables. At single-token decode, a FlashAttention-style CuTe kernel (Dao et al., 2022; NVIDIA, 2026) reads code tensors $\mathbf{C}_K, \mathbf{C}_V$ from paged 32-bit words and computes

$$\mathbf{o} = \text{softmax}\left(\frac{\mathbf{q} \mathcal{D}_K(\mathbf{C}_K)^\top}{\sqrt{d_h}}\right) \mathcal{D}_V(\mathbf{C}_V).$$

Lane-distributed lookup and affine rescaling occur in registers; reconstructed K/V pairs populate Tensor Memory (TMEM) operand tiles (Zadouri et al., 2026) for fused tensor-core scoring, online softmax, and value accumulation, avoiding dense K/V materialization in HBM or shared memory.

4 Experimental Results

4.1 Setup

Backbone LLMs. We evaluate Llama-3.1-8B-Instruct (Grattafiori et al., 2024), Ministral-3-14B-Instruct-2512-BF16 (Liu et al., 2026), and Qwen3-30B-A3B-Instruct-2507 (Yang et al., 2025; Qwen Team, 2025). These instruction-tuned checkpoints span dense and MoE backbones with distinct GQA layouts; all experiments are text-only, and Appendix F gives exact layer and KV-head counts.

Baselines. We compare against Native (full precision), KV-FP8, and TurboQuant-KV4 (Zandieh et al., 2026). Native tests whether cache-side learning can surpass the unmodified checkpoint, while KV-FP8 and TurboQuant-KV4 provide deployed 8-bit and 4-bit references. Appendix E.4 adds roughly matched-effective-bit KIVI and KVQuant controls so that accuracy comparisons are not limited to methods with different cache budgets.

Evaluation tasks. We report held-out PPL (Jelinek et al., 1977) on **The Pile** (Gao et al., 2020) and

seven downstream evaluations using the LM Evaluation Harness (Biderman et al., 2024): LongBench-v1 (Bai et al., 2024), CoQA (Reddy et al., 2019), MATH Maj@1 (Hendrycks et al., 2021b), MMLU (Hendrycks et al., 2021a), AGIEval (Zhong et al., 2024), GPQA Diamond (Rein et al., 2024), and MATH CoT (Hendrycks et al., 2021b). Appendix D additionally reports RULER NIAH (Hsieh et al., 2024).

Implementation details. Table 2 summarizes the default training and quantization setup. Quantizer parameters are shared across tokens at the granularity in Table 2; Appendix F gives parameter counts, metadata sizes, and effective-bit calculations. All quality evaluations and packed-cache inference use the same hard quantization semantics. We implement serving as a custom vLLM attention backend (Kwon et al., 2023), retaining vLLM scheduling and paged KV-cache management; Appendix A gives the chat wrapping recipe. Training uses two NVIDIA B300 GPUs (Aubrey and Stam, 2025); Table 3 includes wall-clock time through final evaluation.

4.2 Accuracy and Deployment Efficiency

Perplexity. Table 3 shows that quantizer-only training reduces PPL from initialization by 3.61%, 2.93%, and 10.74% for Llama, Ministral, and Qwen, and beats Native by 2.2%, 2.8%, and 7.0%, respectively. These gains go beyond recovering initialization error and show that language-model loss can select hard discretizations that local reconstruction objectives would miss. Figure 5 sweeps effective cache bit-width on Llama-3.1-8B-Instruct. Among the 4-bit settings, group 32 gives the lowest PPL (5.38) at 5.00 effective bits; groups 64 and 128 reduce metadata overhead to 4.50 and 4.25 effective bits but raise PPL to 5.44 and 5.51, so we retain group 32.

Downstream accuracy. Against Native, KV-FP8 (8.00 effective bits), and TurboQuant-KV4 (≈ 4.19 effective bits), Table 1 shows that **Beyond** (5.00 effective bits) has the best Avg21 for every model: 44.9, 47.4, and 50.2, beating Native by 1.5, 2.0, and 1.7 points. Across all three models, it improves both code-completion tasks (LCC and RepoBench-P) and the GovReport, QMSum, and MultiNews summarization tasks over Native. Figures 2 and 6 further show consistent CoQA and AGIEval gains, while MATH, MMLU, and GPQA-D remain model-dependent; Appendix D gives all scores. Llama regresses most on PCount and

Model	Method	NQA	Qasper	MF-en	MF-zh	Hotpot	2Wiki	Musique	DuR.	Gov.	QMSum	MNews	VCSUM	TREC	Trivia	SAM	LSHT	PCount	PR-en	PR-zh	LCC	Repo	Avg
Llama-3.1-8B-Instruct	Native	30.3	44.9	56.3	60.6	58.2	49.7	32.1	33.3	34.6	25.3	26.7	17.5	70.0	92.2	22.3	0.5	10.6	100.0	96.0	24.9	25.2	43.4
	KV-FP8	31.0	44.0	56.8	60.7	57.3	50.1	30.0	33.3	34.4	25.3	26.8	17.2	69.0	89.8	20.2	0.5	7.2	99.5	96.5	24.4	23.9	42.8
	TurboQuant	30.9	39.9	48.3	53.0	59.0	49.3	31.9	33.9	34.9	25.4	27.0	17.6	68.5	91.6	22.0	0.5	9.7	99.5	97.5	21.8	24.7	42.2
	Beyond	31.5	47.4	56.2	56.9	53.2	47.4	30.9	<u>33.4</u>	36.7	26.1	28.0	15.8	66.0	90.3	41.5	24.6	4.5	97.5	75.7	42.3	36.9	44.9
Ministral-3-14B-Instruct-2512-BF16	Native	32.4	47.0	48.4	61.8	65.3	56.6	45.4	18.6	22.0	17.2	19.4	13.3	68.7	92.3	36.2	3.5	8.5	100.0	99.5	58.7	37.9	45.4
	KV-FP8	33.1	47.6	48.3	61.8	65.0	55.8	46.6	19.0	22.2	17.3	19.6	13.1	69.4	92.6	36.7	3.5	8.5	100.0	99.5	58.5	38.0	45.5
	TurboQuant	33.0	46.8	48.6	62.2	64.7	55.3	45.4	19.0	21.8	17.4	19.3	13.1	69.2	93.2	36.3	2.2	7.0	100.0	99.5	58.1	37.9	45.2
	Beyond	33.7	49.1	51.1	62.4	<u>65.2</u>	57.7	<u>45.5</u>	23.0	23.1	19.2	20.4	13.8	<u>69.2</u>	92.5	42.5	7.0	9.0	100.0	100.0	66.3	45.1	47.4
Qwen3-30B-A3B-Instruct-2507	Native	31.0	43.0	51.1	67.3	63.5	55.9	36.6	24.3	31.4	22.1	23.5	11.3	80.0	91.3	39.1	49.5	11.0	100.0	100.0	44.0	42.7	48.5
	KV-FP8	31.5	42.9	52.0	66.6	63.8	54.9	36.1	24.1	31.5	22.0	23.3	11.1	81.0	91.3	39.4	49.0	10.5	100.0	100.0	39.9	37.5	48.0
	TurboQuant	31.3	35.8	44.6	55.7	60.3	51.9	37.4	24.4	31.0	21.8	23.6	11.1	72.0	89.1	38.8	49.0	13.5	100.0	<u>85.1</u>	40.1	44.1	45.7
	Beyond	27.9	40.8	54.4	63.1	59.0	52.3	29.5	25.8	32.2	23.2	24.3	10.9	<u>80.0</u>	<u>90.6</u>	43.6	50.5	<u>12.5</u>	100.0	100.0	66.6	67.0	50.2

Table 1: **LongBench-v1 per-task results.** Higher is better. Within each model block, bold and underlining mark the best and second-best scores; Avg is the unweighted mean over all 21 tasks.

Setting	Default
Bits / group size	4-bit codes; $G = 32$
Initialization	KV per-token uniform quantization (Eqs. 3–4)
Trainable params.	K/V quantizer R , T only
Quantizer granularity	Layer $\times$ K/V projection $\times$ KV head $\times$ 32-dim group
K/V placement	K post-RoPE; V post-projection
Dataset	The Pile
Train seq. length	8192 tokens
Held-out chunks	150 validation / 150 disjoint evaluation chunks
Global effective batch size	32
Opt/LR	Adam, 7×10^{-4} cosine decay
Boundary window ϵ_{bw}	0.009
Train budget	Max 500 opt. steps
Early stopping	Full validation every 10 opt. steps; patience 3; select best checkpoint
Training / data seed	42

Table 2: **Default training and quantization setup.** Defaults apply unless stated otherwise.

Model	Native	Init	Trained	Δ	Steps	Train Time
Llama-3.1-8B-Instruct	5.50	5.58	5.38	−3.61%	300	43m
Ministral-3-14B-Instruct-2512-BF16	6.77	6.78	6.58	−2.93%	210	47m
Qwen3-30B-A3B-Instruct-2507	5.31	5.54	4.94	−10.74%	220	3h44m

Table 3: **The Pile evaluation perplexity.** Lower is better. “Init” is the 4-bit/group-32 quantizer before training, and “Trained” is the final checkpoint. Δ is the relative PPL reduction from Init to Trained. “Train Time” is wall-clock time from the first training step through final evaluation.

PR-zh, where exact counting or retrieval outputs amplify small generation changes and the latter additionally uses Chinese input. This selectivity is consistent with training only on continuation NLL from the predominantly English Pile rather than downstream objectives. Accordingly, we interpret the aggregate gains as distribution- and task-dependent adaptation, not uniform dominance over Native on every benchmark slice.

Controls and transfer. Appendix E separates NLL adaptation from discretization, tests zero-update cross-corpus transfer, and adds roughly matched-bit controls. On Llama, **Beyond** is the lowest-PPL compressed control, but a parameter-matched con-

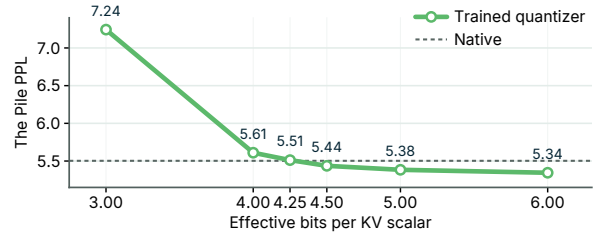


Figure 5: **Effective bit-width versus PPL for Llama-3.1-8B-Instruct.** Points include trained 2-, 3-, 4-, and 5-code-bit/group-32 quantizers and additional 4-code-bit/group-64 and group-128 results, all plotted by effective cache bit-width on the held-out Pile evaluation split. The dashed line is native full-precision PPL. Lower is better.

tinuous adapter is lower, so the gain over Native is not unique to discretization. The trained quantizers improve five of six C4/WikiText-2 pairs; at roughly 5 bits, **Beyond** leads KIVI-R32 and an independent KVQuant reference on LongBench Avg21 and four of six additional benchmarks, although source and rescoring analyses show selective gains and some improved answer-format adherence. These diagnostics separate genuine count or index-binding failures from formatting effects and prevent exact-match gains from being read as a uniform capability improvement.

Deployment performance. We report operator latency separately from end-to-end serving throughput. Figure 7 (top) compares the **Beyond** packed-cache decode kernel with FlashAttention-4 (Zadouri et al., 2026) on one B300 at batches 1 and 16 over 1K–128K contexts; **Beyond** is faster throughout and peaks at $1.50\times$. The bottom plot compares both complete vLLM (Kwon et al., 2023) paths using the official offline driver, cold 1024-token prompts, 128 generated tokens, and the same BF16 backbone. Under this setting, **Beyond**

Method	PPL↓	LB Avg21↑	CoQA↑	MATH Maj@1↑	MMLU↑	AGIEval↑	GPQA-D↑	MATH CoT↑
Full	5.3837	44.89	61.28	45.08	63.94	37.71	30.81	34.12
Raw	5.3905	43.18	51.90	44.72	63.05	36.71	28.79	34.24
Freeze- <i>R</i>	5.4068	44.33	49.03	47.88	62.61	36.20	25.25	34.42
Freeze- <i>T</i>	5.3968	43.56	36.99	45.86	60.95	36.19	27.27	35.52

Table 4: **Component ablation on Llama-3.1-8B-Instruct.** All variants use a frozen backbone, 4-bit/group-32 quantization, and the same evaluation protocol. Full jointly learns reconstruction levels R and assignment thresholds T with half-wave rectification; Raw uses unrectified threshold gradients; Freeze- R learns only T ; and Freeze- T learns only R . LB Avg21 is the mean over 21 LongBench-v1 tasks. Bold marks the best result in each column (lowest PPL and highest task score).

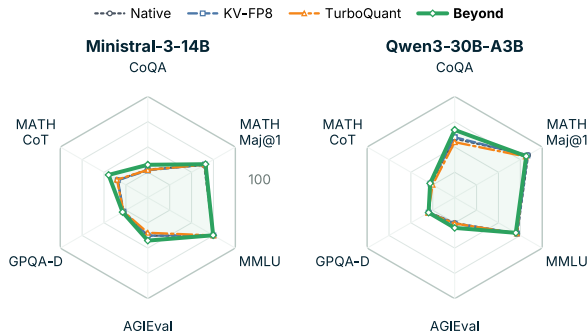


Figure 6: **Six-benchmark comparison on Ministral and Qwen.** Exact scores appear in Appendix D; higher is better.

reaches $1.17\times$ higher throughput at batch 1536. At 0.90 GPU-memory fraction, the largest successful batch grows from 1582 to at least 4500 ($2.84\times$); FlashAttention-4 fails at 1583 and **Beyond** at the next tested point, 4800. The dotted markers are measured full-batch KV-capacity failures rather than extrapolated limits; prefix caching is disabled for both paths.

Component ablation. Table 4 evaluates level learning, threshold learning, and half-wave rectification beyond PPL. Full leads six of eight columns—PPL, LongBench Avg21, CoQA, MMLU, AGIEval, and GPQA-D—while Freeze- R and Freeze- T lead the two MATH metrics. Half-wave rectification improves seven columns over Raw, with only a 0.12-point decrease on MATH CoT. Joint R, T learning beats both frozen variants on PPL and the five non-MATH task aggregates above, while the two MATH reversals confirm that component effects remain task-dependent. Appendix E.6 gives all 21 tasks, a Qwen3-4B ablation, and learning-rate sensitivity.

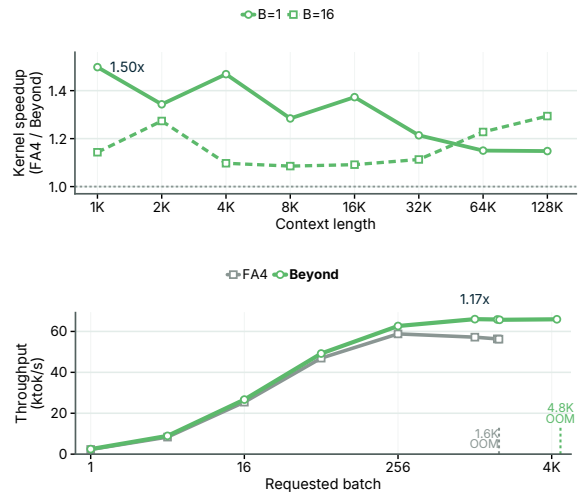


Figure 7: **B300 deployment performance.** Top: attention decode-kernel speedup (FlashAttention-4 latency divided by **Beyond** latency) after 50 warmups and 500 CUDA-graph replays. Bottom: official vLLM offline throughput with cold 1024+128-token requests and an 8192-token scheduling budget; curves stop at the last success, and dotted markers denote the first tested KV-capacity failure.

5 Conclusion

We presented **Beyond**, a loss-aware non-uniform KV-cache quantizer that learns representation levels and thresholds from language-model loss while keeping the forward pass identical to the hard quantizer used at inference. Across three instruction-tuned LLMs, 4-bit/group-32 training beats Native PPL and every tested baseline on LongBench Avg21; the packed-cache B300 path also improves decode latency and end-to-end throughput. Thus, KV compression can provide adaptation benefits beyond memory savings.

Limitations

Beyond studies scalar non-uniform KV-cache quantization with frozen backbone weights under a shared 4-bit/group-32 setup across three instruction-tuned model families, together with an initial effective-bit-width sweep on Llama-3.1-8B-Instruct. Accordingly, “better than full precision” denotes only a measured improvement over the unadapted Native baseline for a reported metric under the stated model, data, task, and quantization setting; it neither establishes an intrinsic advantage of discretization nor guarantees gains over an equally adapted continuous full-precision system. Results vary across tasks and data distributions, and some individual tasks decline, so we regard implicit regularization as a plausible explanation for the aggregate gains rather than an isolated causal effect or universal guarantee. Future work should test additional architectures, bit-widths, grouping choices, and serving configurations, while deployments should independently validate task quality, latency, memory use, and reliability under their target workloads.

Ethical Considerations

This paper presents **Beyond**, a KV-cache quantization method for large language model inference. The intended use is resource-efficient long-context serving: reducing KV-cache memory can lower hardware pressure and improve the practicality of deployment. The method does not modify the underlying model weights or training data, so it should be evaluated under the same responsible-deployment procedures as the base model. In particular, application owners should assess output quality, robustness, privacy, and any domain-specific safety requirements after compression.

References

- Kyle Aubrey and Nick Stam. 2025. [Inside NVIDIA blackwell ultra: The chip powering the AI factory era](#). NVIDIA Technical Blog. Accessed: 2026-05-26.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. [LongBench: A bilingual, multi-task benchmark for long context understanding](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand. Association for Computational Linguistics.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. [Estimating or propagating gradients through stochastic neurons for conditional computation](#). *Preprint*, arXiv:1308.3432.
- Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Hao-nan Li, and 11 others. 2024. [Lessons from the trenches on reproducible evaluation of language models](#). *Preprint*, arXiv:2405.14782.
- Christopher M. Bishop. 1995. [Training with noise is equivalent to tikhonov regularization](#). *Neural Computation*, 7(1):108–116.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2025. [Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling](#). In *COLM 2025*. OpenReview.net.
- Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S. Abdelfattah, and Kai-Chiang Wu. 2025. [Palu: KV-cache compression with low-rank projection](#). In *The Thirteenth International Conference on Learning Representations*. OpenReview.net.
- Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. [Pact: Parameterized clipping activation for quantized neural networks](#). *Preprint*, arXiv:1805.06085.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. [Flashattention: Fast and memory-efficient exact attention with io-awareness](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359.
- Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. 2020. [Learned step size quantization](#). In *8th International Conference on Learning Representations*. OpenReview.net.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. [The Pile: An 800gb dataset of diverse text for language modeling](#). *arXiv preprint arXiv:2101.00027*.
- Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. 2019. [Differentiable soft quantization: Bridging full-precision and low-bit neural networks](#). In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4851–4860.

- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. [The llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.
- Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. 2024. [Zipcache: Accurate and efficient KV cache quantization with salient token identification](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 68287–68307. Curran Associates, Inc.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. [Measuring massive multitask language understanding](#). In *International Conference on Learning Representations*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. [Measuring mathematical problem solving with the MATH dataset](#). In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. [Kvquant: Towards 10 million context length llm inference with kv cache quantization](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 1270–1303. Curran Associates, Inc.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. [RULER: What’s the real context size of your long-context language models?](#) In *First Conference on Language Modeling (COLM)*.
- Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2018. [Quantized neural networks: Training neural networks with low precision weights and activations](#). *Journal of Machine Learning Research*, 18(187):1–30.
- Frederick Jelinek, Robert L. Mercer, Lalit R. Bahl, and James K. Baker. 1977. [Perplexity: A measure of the difficulty of speech recognition tasks](#). *The Journal of the Acoustical Society of America*, 62(S1):S63–S63.
- Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. 2024. [GEAR: An efficient error reduction framework for KV cache compression in LLM inference](#). In *Proceedings of The 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, volume 262 of *Proceedings of Machine Learning Research*, pages 305–321. PMLR.
- Diederik P. Kingma and Jimmy Ba. 2015. [Adam: A method for stochastic optimization](#). In *3rd International Conference on Learning Representations, ICLR 2015*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. [Efficient memory management for large language model serving with PagedAttention](#). In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. [Snapkv: Llm knows what you are looking for before generation](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 22947–22970. Curran Associates, Inc.
- Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2025. [QServe: W4A8KV4 quantization and system co-design for efficient LLM serving](#). In *Proceedings of Machine Learning and Systems*, volume 7. ML-Sys.
- Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024a. [Minicache: Kv cache compression in depth dimension for large language models](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 139997–140031. Curran Associates, Inc.
- Alexander H. Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, et al. 2026. [Ministral 3](#). *Preprint*, arXiv:2601.08584.
- Peiyu Liu, Ze-Feng Gao, Xin Zhao, Yipeng Ma, Tao Wang, and Ji-Rong Wen. 2024b. [Unlocking data-free low-bit quantization with matrix decomposition for KV cache compression](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2430–2440, Bangkok, Thailand. Association for Computational Linguistics.
- Xin Liu, Xudong Wang, Pei Liu, and Guoming Tang. 2025. [Zsmmerge: Zero-shot kv cache compression for memory-efficient long-context llms](#). *Preprint*, arXiv:2503.10714.
- Zechun Liu, Kwang-Ting Cheng, Dong Huang, Eric P. Xing, and Zhiqiang Shen. 2022. [Nonuniform-to-uniform quantization: Towards accurate quantization via generalized straight-through estimation](#). In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022*, pages 4942–4952. IEEE.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024c. [KIVI: A tuning-free asymmetric 2bit quantization for KV cache](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344. PMLR.

- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- NVIDIA. 2026. [CuTe DSL: Introduction](#). NVIDIA CUTLASS Documentation. Accessed: 2026-05-26.
- Qwen Team. 2025. [Qwen3-30B-A3B-Instruct-2507](#). Official model card.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. [Exploring the limits of transfer learning with a unified text-to-text transformer](#). *Journal of Machine Learning Research*, 21(140):1–67.
- Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. 2018. [On the convergence of adam and beyond](#). In *6th International Conference on Learning Representations, ICLR 2018*. OpenReview.net.
- Siva Reddy, Danqi Chen, and Christopher D. Manning. 2019. [CoQA: A conversational question answering challenge](#). *Transactions of the Association for Computational Linguistics*, 7:249–266.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. [GPQA: A graduate-level google-proof Q&A benchmark](#). In *First Conference on Language Modeling*.
- Utkarsh Saxena, Gobinda Saha, Sakshi Choudhary, and Kaushik Roy. 2024. [Eigen attention: Attention in low-rank space for KV cache compression](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15332–15344, Miami, Florida, USA. Association for Computational Linguistics.
- Donghyun Son, Euntae Choi, and Sungjoo Yoo. 2025. [Nsnquant: A double normalization approach for calibration-free low-bit vector quantization of kv cache](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 48151–48186.
- Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. [Roformer: Enhanced transformer with rotary position embedding](#). *Neurocomputing*, 568:127063.
- Yi Su, Yuechi Zhou, Quantong Qiu, Juntao Li, Qingrong Xia, Ping Li, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025. [Accurate KV cache quantization with outlier tokens tracing](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12895–12915, Vienna, Austria. Association for Computational Linguistics.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. [Llama: Open and efficient foundation language models](#). *Preprint*, arXiv:2302.13971.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. 2024. [Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks](#). *Preprint*, arXiv:2407.08454.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. [Efficient streaming language models with attention sinks](#). In *The Twelfth International Conference on Learning Representations*. OpenReview.net.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.
- Dingyu Yao, Chenxu Yang, Zhengyang Tong, Zheng Lin, Wei Liu, Jian Luan, and Weiping Wang. 2026. [VecInfer: Efficient LLM inference with low-bit KV cache via outlier-suppressed vector quantization](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31527–31543, San Diego, California, United States. Association for Computational Linguistics.
- Zhihang Yuan, Yuzhang Shang, Yue Song, Dawei Yang, Qiang Wu, Yan Yan, and Guangyu Sun. 2023. [Asvd: Activation-aware singular value decomposition for compressing large language models](#). *Preprint*, arXiv:2312.05821.
- Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. 2026. [Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling](#). *Preprint*, arXiv:2603.05451.
- Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2026. [TurboQuant: Online vector quantization with near-optimal distortion rate](#). In *The Fourteenth International Conference on Learning Representations*. OpenReview.net.
- Tianyi Zhang, Jonah Yi, Zhaozhuo Xu, and Anshumali Shrivastava. 2024a. [Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 3304–3331. Curran Associates, Inc.
- Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. 2024b. [CaM: Cache merging for memory-efficient LLMs inference](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 58840–58850. PMLR.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuan-dong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. [H2O: Heavy-hitter oracle for efficient generative inference of large language models](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710. Curran Associates, Inc.

Wanjuan Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. 2024. [AGIEval: A human-centric benchmark for evaluating foundation models](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2299–2314, Mexico City, Mexico. Association for Computational Linguistics.

A Chat-Prefix Wrapping for Pile Training

For the instruct-model runs, plain Pile documents are formatted as assistant continuations under the model’s own tokenizer chat template. The fixed user message is:

Continue the following text.

Let p denote this prompt. We render this one-turn prompt with an assistant-generation prefix using the tokenizer template of each base model, and then append the raw Pile document text as the target continuation. Equivalently, the input sequence has the form

$$\text{ChatPrefix}_\theta(p) \parallel x,$$

where x is the Pile document text and ChatPrefix_θ denotes the model-specific chat-template prefix before assistant content. The prefix tokens are included in the context but are masked out of the loss; only the appended document tokens contribute to the next-token NLL used to train R and T . The same wrapping recipe is used for the training stream and the held-out Pile validation/evaluation chunks.

B Training Dynamics

Figure 8 plots the optimization dynamics for the three 4-bit/group-32 runs reported in Table 3. The curves show the validation loss recorded periodically during training.

C Boundary-Window Sensitivity

Table 5 reports additional Llama-3.1-8B-Instruct runs that vary the boundary window ϵ_{bw} used for threshold pseudo-gradients. Both runs use the same 4-bit/group-32 setup and Pile evaluation protocol as Table 3. The results are close over the tested range, while the default $\epsilon_{\text{bw}} = 0.009$ reaches 5.38 PPL.

ϵ_{bw}	Eval. PPL	Stopped at
0.007	5.40	step 150
0.009	5.38	step 300
0.011	5.39	step 360

Table 5: **Boundary-window sensitivity on Llama-3.1-8B-Instruct.** PPL is reported on the same Pile evaluation split as Table 3. Lower is better.

D Detailed Benchmark Results

Table 6 reports exact aggregate scores for CoQA, MATH Maj@1, MMLU, AGIEval, GPQA-D, and

MATH CoT. Figure 9 separately visualizes RULER NIAH (Hsieh et al., 2024) for Native and **Beyond** across all three models.

E Additional Experimental Analyses

This section reports the additional controls and diagnostics summarized in Sec. 4. Unless stated otherwise, the backbone is frozen and only cache-side parameters are updated.

E.1 Cache-Side Objective and Parameterization Controls

Table 7 separates the effects of compression, parameterization, and the training objective on Llama-3.1-8B-Instruct. All learned controls use the same Pile split, 500-step ceiling, and early-stopping rule, but early stopping produces unequal token exposure. Each optimizer step contains $32 \times 8,192$ input tokens; “best/run” denotes the exposure at the restored-best checkpoint and over the complete run, respectively.

The uniform-affine control learns endpoints a and b , from which all levels and thresholds are derived:

$$r_i = a + (b - a)i/15, \quad t_i = (r_i + r_{i+1})/2.$$

These endpoints are global trainable variables for each physical quantizer group, rather than the token-group min and scale computed from each activation. The reconstruction control and **Beyond** instead learn the same independent variables $r_{0:15}$ and $t_{0:14}$; the former minimizes hard-QDQ K/V MSE locally (K after RoPE and V after projection), whereas **Beyond** minimizes next-token NLL through the frozen model.

The continuous control matches the 31-scalar parameter count without quantization or an STE. For group-normalized u , $z = 2u - 1$, and first-kind Chebyshev polynomials T_k , it applies

$$x' = x + \text{span}(x)u(1 - u) \tanh f(z),$$

$$f(z) = \sum_{k=0}^{30} c_k T_k(z),$$

where $T_0(z) = 1$, $T_1(z) = z$, and $T_k(z) = 2zT_{k-1}(z) - T_{k-2}(z)$. Initializing $c_{0:30} = 0$ makes the map exactly the identity, after which NLL updates only these coefficients. Thus, **Beyond** is the strongest compressed control in Table 7, but the continuous adapter’s lower PPL shows that Native-to-**Beyond** gains do not isolate an intrinsic advantage of discretization.

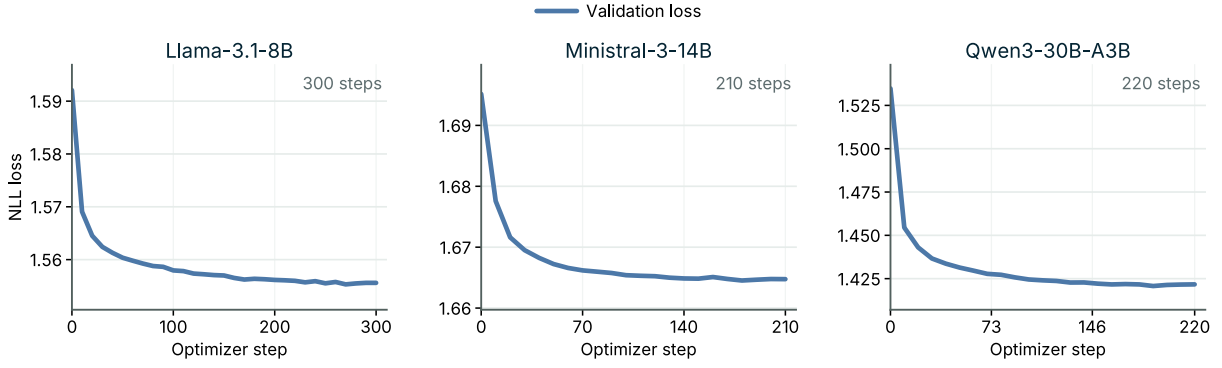


Figure 8: **Training dynamics for the three evaluated models.** Curves show validation NLL evaluated every 10 optimizer steps.

Model	Method	CoQA	MATH Maj@1	MMLU	AGIEval	GPQA-D	MATH CoT
Llama-3.1-8B	Native	<u>23.6</u>	47.4	<u>63.0</u>	<u>36.3</u>	<u>29.3</u>	40.0
	KV-FP8	21.9	<u>46.9</u>	62.8	36.2	27.8	40.0
	TurboQuant	21.4	45.1	62.6	35.9	26.8	<u>38.2</u>
	Beyond	61.3	45.1	63.9	37.7	30.8	34.1
Ministral-3-14B	Native	<u>27.6</u>	64.4	75.0	<u>37.7</u>	<u>27.8</u>	34.5
	KV-FP8	26.9	<u>65.4</u>	<u>74.9</u>	<u>37.7</u>	<u>27.8</u>	34.2
	TurboQuant	27.3	65.1	75.0	34.8	<u>27.8</u>	<u>35.3</u>
	Beyond	32.4	66.0	74.7	42.6	28.8	44.8
Qwen3-30B-A3B	Native	58.3	83.9	71.4	25.3	29.3	27.2
	KV-FP8	<u>59.6</u>	<u>83.1</u>	<u>71.2</u>	<u>25.7</u>	<u>29.8</u>	<u>27.7</u>
	TurboQuant	54.8	81.1	71.0	25.6	30.3	25.2
	Beyond	66.8	81.7	69.8	30.0	<u>29.8</u>	28.0

Table 6: **Detailed non-LongBench-v1 Benchmark Scores.** Scores are percentages where higher is better. Within each model block, the best score is bolded and the second-best score is underlined.

Method	Variables/G32	Objective; cache	Best/run tokens	PPL↓
Native	None	None; uncompressed	–	5.5030
Uniform-affine	2 endpoints	NLL; uniform B4/G32	47.2M/55.1M	5.5125
Non-uniform recon.	16 levels + 15 thresholds	Local K/V MSE; B4/G32	15.7M/23.6M	5.5686
Beyond	16 levels + 15 thresholds	NLL; B4/G32	76.0M/83.9M	<u>5.3837</u>
Continuous adapter	31 Chebyshev coefficients	NLL; uncompressed	131.1M/131.1M	5.3406

Table 7: **Cache-side adaptation controls on Llama-3.1-8B-Instruct.** Lower PPL is better. Bold marks the overall best result; underlining marks the best compressed control. Token exposures differ because all runs use early stopping rather than a matched realized-token budget.

E.2 Zero-Update Cross-Corpus Transfer

We evaluate each final Pile-trained B4/G32 quantizer on C4 (Raffel et al., 2020) and WikiText-2 (Merity et al., 2017) without target-corpus updates, following the KVQuant 8,192-token PPL protocol. Table 8 shows positive transfer in five

of six model–corpus pairs. Llama and Qwen improve on both corpora, while Ministral improves on WikiText-2 and is the sole C4 exception, increasing PPL by 0.0126.

E.3 Source-Specific Adaptation

To probe distribution dependence, we freeze Llama and train only its B4/G32 quantizer on six additional sources. Every source-matched held-out PPL in Table 9 is below the corresponding Native value, but the downstream effects in Table 10 follow the source rather than improving uniformly.

The multilingual quantizer also partially recovers two weak Pile-trained slices: PCount rises from 4.50 to 5.03 and PR-zh from 75.70 to 88.13, compared with Native scores of 10.58 and 96.04. These results support source-aligned adaptation and cross-task transfer, but not a claim of uniform improve-



Figure 9: **RULER NIAH heatmaps over depth and word count.** Each panel uses the 500 max-context NIAH samples, bins them by needle depth, and tiles the resulting depth-bin retrieval score across word-count bins to preserve the full KIVI-style heatmap layout.

Model	C4 Native↓	C4 Beyond↓	WikiText-2 Native↓	WikiText-2 Beyond↓
Ministral-3-14B-Instruct	6.6637	6.6763	5.6177	5.6147
Llama-3.1-8B-Instruct	8.0224	7.9328	6.4993	6.4502
Qwen3-30B-A3B-Instruct	7.1080	6.9129	6.5423	6.0616

Table 8: **Zero-update cross-corpus PPL.** The quantizers are trained only on The Pile and evaluated unchanged. Bold marks the lower PPL within each Native–Beyond pair.

Training/evaluation source	Native↓	Beyond↓
FineWeb	14.37	11.76
OpenWebMath	5.72	5.42
Chinese FineWeb2	20.93	16.94
Multilingual FineWeb2	11.03	10.86
HotpotQA + 2WikiMQA	1.97	1.38
UltraChat	2.99	2.77

Table 9: **Source-matched held-out PPL for Llama-3.1-8B-Instruct.** The backbone is frozen and only the B4/G32 quantizer is trained. Bold marks the lower PPL in each row.

ment.

E.4 Roughly Matched Effective-Bit Accuracy References

Table 11 compares Llama accuracy at a roughly matched effective 5-bit KV-cache payload. **Beyond** uses one dense B4/G32 cache. KIVI-R32 (Liu et al., 2024c) uses K4/V4 with G32 and an FP16 residual window of 32 tokens, while our independent KVQuant (Hooper et al., 2024) reference uses Fisher-weighted non-uniform quantization, one sink token, and retained FP16 outliers.

Beyond has the highest LongBench Avg21 and exceeds both compressed references on four of the six additional benchmarks. The cache structures nevertheless differ: KIVI periodically migrates its FP16 residual into quantized history, whereas KVQuant requires indexed sparse-outlier access beside dense codes. We therefore do not infer de-

Quantizer comparison	Evaluation slice	Change
OpenWebMath vs. FineWeb	MATH Maj@1 / MATH CoT	+4.56 / +3.26 points
Chinese FineWeb2 vs. FineWeb	Mean over five Chinese tasks	+3.72 points
Multilingual FineWeb2 vs. FineWeb	Mean over four bilingual retrieval/QA tasks	+12.44 points
HotpotQA + 2WikiMQA vs. Native	HotpotQA/2WikiMQA/MuSiQue mean	46.68 → 49.41
HotpotQA + 2WikiMQA vs. Native	MuSiQue (unseen during adaptation)	32.13 → 34.84
UltraChat vs. Native	CoQA	23.65 → 57.63
UltraChat vs. Native	LongBench Avg21	43.39 → 41.92

Table 10: **Source-aligned downstream effects.** The five-task Chinese mean covers MultiFieldQA-zh, DuReader, VCSum, LSHT, and PassageRetrieval-zh; the four-task bilingual mean covers the English and Chinese versions of MultiFieldQA and PassageRetrieval.

Method	LB Avg21↑	CoQA↑	MATH Maj@1↑	MMLU↑	AGIEval↑	GPQA-D↑	MATH CoT↑
Native	43.39	23.65	47.40	63.00	36.30	29.29	40.00
Pile-trained Beyond (B4/G32)	44.89	61.28	45.08	63.94	37.71	30.81	34.12
KIVI-R32 (K4/V4, G32)	43.15	23.62	46.84	63.08	36.03	26.26	40.06
KVQuant-Sink1-Sparse5bit	42.06	21.18	47.06	62.74	35.87	28.28	39.44

Table 11: **Roughly matched effective-bit accuracy references on Llama-3.1-8B-Instruct.** Bold marks the highest score in each column. These are accuracy references rather than a latency comparison because the methods use different residual and sparse side paths.

ployment speed from Table 11. We omit a full-suite Coupled Quantization row because complete author code for an equivalent evaluation is unavailable; GEAR’s low-rank/sparse residual recovery is orthogonal to the base scalar quantizer and is potentially composable with **Beyond**.

E.5 Score and Formatting Diagnostics

Exact-match tasks can turn small generation changes into large score changes. For PCount, 13 of the 14 Native-to-**Beyond** losses replace the exact Native count with a larger integer near miss; the 10.58-to-4.50 score change corresponds to about 12 fewer exact answers among 200 examples. For PR-zh, among 48 losses, 29 select the wrong passage index and 19 begin with the correct label but append text containing additional numbers, which the exact scorer penalizes. These are substantive count/index-binding failures, not only cosmetic formatting errors.

Formatting nevertheless explains part of the positive movement on other tasks. In the 500-example CoQA diagnostic set, outputs beginning with Q : fall from 108 to 47; rescoring the generated line closest to the reference still leaves **Beyond** ahead by 14.98 points. For the code tasks, stripping prose and scoring the first fenced code line changes **Beyond** minus Native to +9.13 on LCC and −2.56 on RepoBench-P. Thus, improved answer-format adherence contributes to some reported gains, but

neither fully explains CoQA nor yields a uniform capability improvement across code tasks.

E.6 Additional Component Results

Table 12 expands the aggregate ablation in Table 4 across all 21 LongBench-v1 tasks. The camera-ready PPL values remain 5.3837/5.3905/5.4068/5.3968 for Full/Raw/Freeze-*R*/Freeze-*T*, respectively. The per-task reversals reinforce that jointly learning levels and thresholds improves the aggregate without dominating every task.

The Qwen3-4B-Instruct companion gives PPL 5.9828/6.2489/6.0126 for Full/Freeze-*R*/Freeze-*T*; Full is best in this three-way comparison. The Llama learning-rate sweep gives PPL 5.3871, **5.3837**, and 5.3958 at 5×10^{-4} , 7×10^{-4} , and 9×10^{-4} , respectively, supporting the default 7×10^{-4} within the tested range. Bit-width and group-size sweeps in Fig. 5 use the Full method rather than the component variants.

F Quantizer Granularity and Storage

Beyond stores one independently parameterized scalar quantizer table for each layer, projection type $a \in \{K, V\}$, KV head, and head-dimension group of size $G = 32$. The table is shared across sequence positions and contains $K = 2^b = 16$ representation levels R and $K - 1 = 15$ thresholds T . The runtime accepts independently non-uniform

Task	Full	Raw	Freeze- R	Freeze- T
NarrativeQA	31.49	31.08	29.76	29.53
Qasper	47.40	48.65	48.04	47.38
MultiFieldQA-en	56.24	54.00	54.23	53.10
MultiFieldQA-zh	56.91	57.54	58.44	58.29
HotpotQA	53.17	53.74	53.97	55.25
2WikiMQA	47.39	47.32	46.88	48.48
MuSiQue	30.87	27.68	26.42	30.91
DuReader	33.36	34.07	31.23	32.00
GovReport	36.67	37.01	36.48	36.82
QMSum	26.14	26.31	26.04	26.80
MultiNews	28.02	28.01	28.04	27.33
VCSum	15.84	16.20	16.62	16.41
TREC	66.00	59.00	65.00	66.50
TriviaQA	90.31	89.71	90.93	90.35
SAMSum	41.46	33.50	41.41	41.53
LSHT	24.60	16.50	29.00	11.67
PassageCount	4.50	5.00	4.91	4.48
PassageRetrieval-en	97.50	98.50	99.00	97.17
PassageRetrieval-zh	75.70	75.43	85.89	77.50
LCC	42.28	35.58	29.53	31.30
RepoBench-P	36.90	31.93	29.10	31.89

Table 12: **Per-task LongBench-v1 component ablation.** Bold marks the highest score in each row.

Model	L	H_{kv}	Tables/proj.	$ R $	$ T $	$ \varphi $	FP16 mem.
Llama-3.1-8B-Instruct	32	8	32	32,768	30,720	63,488	126,976 B (124 KiB)
Minstral-3.14B-Instruct	40	8	32	40,960	38,400	79,360	158,720 B (155 KiB)
Qwen3-30B-A3B-Instruct	48	4	16	24,576	23,040	47,616	95,232 B (93 KiB)

Table 13: **Learned quantizer parameter counts and persistent table memory.** Counts are from the final 4-bit/group-32 quantizer configs. Training retains FP32 master parameters, whereas deployed R and T tensors use IEEE FP16. This excludes per-token cache codes and min/scale statistics, which are counted in the cache bit-width calculation below.

K and V tables. In the retained final artifacts, all Llama and Qwen K/V tables and all Minstral V tables are non-uniform; Minstral K remained exactly on its initialized uniform grid and is reported without post-hoc perturbation. All three evaluated configurations have $d_h = 128$. For a model with L layers, H_{kv} KV heads, and head dimension d_h , the number of learned tables is

$$N_{\text{tab}} = 2LH_{kv} \frac{d_h}{G},$$

where the factor of 2 accounts for K and V. Thus

$$\begin{aligned} |R| &= N_{\text{tab}}K, \\ |T| &= N_{\text{tab}}(K-1), \\ |\varphi| &= N_{\text{tab}}(2K-1). \end{aligned}$$

At deployment, thresholds and reconstruction levels both use IEEE FP16 storage. Thresholds assign newly produced K/V states to cache codes, while the CuTe attention kernel resolves packed codes through the corresponding FP16 reconstruction-level LUT. FP32 masters remain a training-only optimizer representation.

The packed cache stores $b = 4$ code bits per KV scalar plus two group statistics, min and scale, for every group of $G = 32$ scalars. With 16-bit statistics, the cache storage per KV scalar is

$$b_{\text{cache}} = \frac{Gb + 2 \cdot 16}{G} = 4 + \frac{32}{32} = 5.0 \text{ bits.}$$

Equivalently, the packed cache has a $16/5 = 3.2\times$ storage reduction relative to a 16-bit dense KV cache, before allocator/page padding. If the persistent FP16 quantizer tables are amortized over a batch of B sequences with cache length S , the amortized bit-width is

$$\begin{aligned} b_{\text{amort}}(B, S) &= b_{\text{cache}} + \frac{16|\varphi|}{2BLSH_{kv}d_h} \\ &= 5 + \frac{15.5}{BS} \text{ bits per KV scalar,} \end{aligned}$$

where the final equality uses $b = 4$, $K = 16$, and $G = 32$. For a single sequence, this gives 5.0019 bits at $S = 8192$ and 5.0005 bits at $S = 32768$; larger batches amortize the learned tables further.

For the KV-FP8 baseline, the run uses vLLM `kv_cache_dtype=fp8`; the cache payload stores one 8-bit FP8 value per K/V scalar. If we also count the two optional FP32 K/V scale scalars per layer, the amortized bit-width is

$$\begin{aligned} b_{\text{amort}}^{\text{FP8}}(B, S) &= 8 + \frac{2L \cdot 32}{2BLSH_{kv}d_h} \\ &= 8 + \frac{32}{BSH_{kv}d_h}. \end{aligned}$$

This is measured in bits per KV scalar; the additive term is negligible for the evaluated context lengths.

G Mathematical Details and Proofs

G.1 KV Quantization as Implicit Regularization

This subsection proves the perturbation bounds used in Sec. 3.2. The argument is intentionally local: it identifies the loss terms that any downstream-aware quantizer must control, rather than proving that every quantizer improves every model.

Proof of Proposition 3.1. Fix a perturbation $d = \delta_\varphi$ and restrict the loss to the line segment from z to $z + d$ by defining $f(t) = \Psi(z + td)$ for $t \in [0, 1]$. Then $f'(0) = \langle g, d \rangle$ and $f''(t) = d^\top \nabla^2 \Psi(z + td) d$. Applying the second-order Taylor formula with integral remainder to this one-dimensional function at $t = 0$ gives

$$f(1) = f(0) + f'(0) + \frac{1}{2}f''(0) + \int_0^1 (1-t)(f''(t) - f''(0)) dt.$$

Here the integral remainder is the part not captured by the Hessian at the base point z : it measures how $f''(t)$, equivalently the Hessian of Ψ along the segment from z to $z + d$, changes relative to $f''(0)$. Substituting back $f(1) = \Psi(z + d)$, $f(0) = \Psi(z)$, $f'(0) = d^\top H d$, and $H = \nabla^2 \Psi(z)$ yields

$$\Psi(z + d) = \Psi(z) + \langle g, d \rangle + \frac{1}{2}d^\top H d + r(d),$$

where

$$r(d) = \int_0^1 (1-t) d^\top (\nabla^2 \Psi(z + td) - H) d dt.$$

The Lipschitz-Hessian condition $\|\nabla^2 \Psi(z + u) - \nabla^2 \Psi(z)\|_{\text{op}} \leq \beta \|u\|_2$ gives, with $\Delta_t = \nabla^2 \Psi(z + td) - H$,

$$\begin{aligned} |r(d)| &\leq \int_0^1 (1-t) \|d\|_2^2 \|\Delta_t\|_{\text{op}} dt \\ &\leq \int_0^1 (1-t) \|d\|_2^2 \beta t \|d\|_2 dt \\ &= \beta \|d\|_2^3 \int_0^1 t(1-t) dt = \frac{\beta}{6} \|d\|_2^3. \end{aligned}$$

The first inequality uses $|d^\top A d| \leq \|d\|_2 \|A d\|_2 \leq \|A\|_{\text{op}} \|d\|_2^2$ with $A = \Delta_t$. The second uses the Lipschitz condition with $u = td$, giving $\|\Delta_t\|_{\text{op}} \leq \beta t \|d\|_2$. The factor $1/6$ is the scalar integral $\int_0^1 t(1-t) dt = 1/6$. Taking expectation yields

$$\begin{aligned} \mathbb{E} \Psi(z + \delta_\varphi) &= \Psi(z) + \langle g, \mathbb{E} \delta_\varphi \rangle \\ &\quad + \frac{1}{2} \text{Tr} \left(H \mathbb{E} [\delta_\varphi \delta_\varphi^\top] \right) + \rho_\varphi, \end{aligned}$$

with $|\rho_\varphi| \leq \frac{\beta}{6} \mathbb{E} \|\delta_\varphi\|_2^3$. For token NLL, let $s = h(z)$ be the downstream logits, $\pi = \text{Softmax}(s)$, J_h the Jacobian of h , and $\mathcal{F}_\pi = \text{Diag}(\pi) - \pi \pi^\top \succeq 0$. The exact downstream Hessian decomposes as

$$H = J_h^\top \mathcal{F}_\pi J_h + \sum_j (\pi_j - \mathbf{1}\{j = y\}) \nabla^2 h_j(z).$$

Thus the generalized Gauss–Newton/Fisher component $H_{\text{GGN}} = J_h^\top \mathcal{F}_\pi J_h$ is positive semidefinite, whereas the downstream-network second-order term can make H indefinite. Fisher-weighted distortion is obtained only after omitting the linear term and replacing H by a fixed H_{GGN} , which gives $\frac{1}{2} \mathbb{E} [\delta_\varphi^\top H_{\text{GGN}} \delta_\varphi] = \frac{1}{2} \text{Tr}(H_{\text{GGN}} C_\varphi)$. Plain reconstruction follows only in the isotropic case $H_{\text{GGN}} = \lambda I$ with $\lambda > 0$, where this term becomes $\frac{\lambda}{2} \mathbb{E} \|\delta_\varphi\|_2^2$. Hence reconstruction error is not the general downstream objective. $\square$

Setup. For one attention head, define

$$\begin{aligned} a &= \frac{1}{\sqrt{d}} q K^\top \in \mathbb{R}^S, \\ p &= \text{Softmax}(a), \\ o &= pV. \end{aligned}$$

Quantization induces perturbations $\hat{K} = K + \Delta K$ and $\hat{V} = V + \Delta V$. The key perturbation creates logit noise

$$\begin{aligned} \varepsilon &\triangleq \frac{1}{\sqrt{d}} q (\Delta K)^\top, \\ a' &= a + \varepsilon, \\ p' &= \text{Softmax}(a + \varepsilon), \\ o' &= p' \hat{V}. \end{aligned}$$

The mean perturbation $\mu = \mathbb{E}[\Delta o]$ may be nonzero for a learned quantizer; this is the degree of freedom that lets loss-aware quantization act differently from reconstruction-only, approximately zero-mean noise.

Assumption G.1 (Small moment-controlled KV perturbations). Assume ΔK and ΔV have bounded second moments and are small enough for the first-order attention expansion to dominate the second-order remainder. Assume further that either ΔK and ΔV are independent, or their cross-covariance terms are negligible at first order.

Lemma G.2 (Softmax first-order expansion). Let $p = \text{Softmax}(a)$. For small ε ,

$$\begin{aligned} p' &= p + J_{\text{sm}}(a) \varepsilon + O(\|\varepsilon\|^2), \\ J_{\text{sm}}(a) &= \text{Diag}(p) - p p^\top. \end{aligned}$$

Proof. Taylor expand the smooth map Softmax around a and use its standard Jacobian. $\square$

Lemma G.3 (Linearized attention output perturbation). *Let*

$$\Delta o \triangleq (J_{\text{sm}}(a)\varepsilon)V + p\Delta V.$$

Then, up to second-order terms,

$$o' = o + \Delta o + O(\|\varepsilon\|^2 + \|\varepsilon\| \|\Delta V\|).$$

Proof. Expand

$$\begin{aligned} o' &= p'(V + \Delta V) \\ &= pV + (p' - p)V + p\Delta V \\ &\quad + (p' - p)\Delta V. \end{aligned}$$

Using Lemma G.2, the first-order terms are exactly $(J_{\text{sm}}(a)\varepsilon)V + p\Delta V$, while $(p' - p)\Delta V$ and the Taylor remainder of p' are second order. $\square$

Proof of Proposition 3.2. Let $o_{\text{lin}} = o + \Delta o$ be the linearized attention output from Lemma G.3. By L -smoothness of ℓ ,

$$\ell(o_{\text{lin}}) \leq \ell(o) + \langle \nabla \ell(o), \Delta o \rangle + \frac{L}{2} \|\Delta o\|_2^2.$$

Taking expectation gives

$$\mathbb{E}[\ell(o_{\text{lin}})] \leq \ell(o) + \langle \nabla \ell(o), \mathbb{E}[\Delta o] \rangle + \frac{L}{2} \mathbb{E} \|\Delta o\|_2^2,$$

which proves the bound with $\mu = \mathbb{E}[\Delta o]$. If $\langle \nabla \ell(o), \mu \rangle < -\frac{L}{2} \mathbb{E} \|\Delta o\|_2^2$, then the right-hand side is strictly smaller than $\ell(o)$, so the expected quantized loss is lower than the full-precision local loss. $\square$

Corollary G.4 (Zero-mean quantization gives a second-order regularizer). *Under Assumption G.1, if additionally $\mathbb{E}[\Delta K] = 0$ and $\mathbb{E}[\Delta V] = 0$, then $\mu = 0$ in the first-order model, and Proposition 3.2 reduces to*

$$\mathbb{E}[\ell(o_{\text{lin}})] \leq \ell(o) + \frac{L}{2} \mathbb{E} \|\Delta o\|_2^2.$$

Thus zero-mean quantization behaves locally like an input-dependent second-order penalty.

Proof. Since $\mathbb{E}[\Delta K] = 0$ and $\mathbb{E}[\Delta V] = 0$, $\mathbb{E}[\varepsilon] = 0$ and $\mathbb{E}[\Delta o] = (J_{\text{sm}}(a)\mathbb{E}[\varepsilon])V + p\mathbb{E}[\Delta V] = 0$ at first order. Substituting $\mu = 0$ into Proposition 3.2 gives the result. $\square$

Proof of Proposition 3.3. Let $w_j^\top$ denote the j -th row of W and $\pi_t = \text{Softmax}(W o_t)$. The cross-entropy loss at position t is

$$\begin{aligned} \ell_t(o_t) &= -\log \pi_{t,y_t} \\ &= -w_{y_t}^\top o_t + \log \sum_k \exp(w_k^\top o_t). \end{aligned}$$

Therefore

$$\begin{aligned} \nabla_{o_t} \ell_t(o_t) &= -w_{y_t} + \frac{\sum_j \exp(w_j^\top o_t) w_j}{\sum_k \exp(w_k^\top o_t)} \\ &= \sum_j \pi_{t,j} w_j - w_{y_t}. \end{aligned}$$

Since $\sum_j \pi_{t,j} = 1$,

$$\begin{aligned} \nabla_{o_t} \ell_t(o_t) &= \sum_j \pi_{t,j} (w_j - w_{y_t}) \\ &= \sum_{j \neq y_t} \pi_{t,j} (w_j - w_{y_t}). \end{aligned}$$

Taking the inner product with μ_t gives the linear term in the loss change,

$$\ell_t(o_t + \mu_t) - \ell_t(o_t) = \langle \nabla_{o_t} \ell_t(o_t), \mu_t \rangle + O(\|\mu_t\|_2^2),$$

where

$$\langle \nabla_{o_t} \ell_t(o_t), \mu_t \rangle = \sum_{j \neq y_t} \pi_{t,j} \langle w_j - w_{y_t}, \mu_t \rangle.$$

Now define the one-step margin shift for any competing token $j \neq y_t$:

$$\begin{aligned} \Delta m_t(j) &\triangleq [s_j(o_t + \mu_t) - s_{y_t}(o_t + \mu_t)] \\ &\quad - [s_j(o_t) - s_{y_t}(o_t)]. \end{aligned}$$

Because $s_j(o) = w_j^\top o$,

$$\Delta m_t(j) = \langle w_j - w_{y_t}, \mu_t \rangle.$$

Substituting this identity into the loss expression yields

$$\langle \nabla_{o_t} \ell_t(o_t), \mu_t \rangle = \sum_{j \neq y_t} \pi_{t,j} \Delta m_t(j).$$

Thus cross-entropy averages the one-step competing-margin shifts, weighted by the model probabilities.

For a fixed competing continuation $c_{1:T}$, keep the teacher-forced states o_t fixed from the target

prefixes $y_{<t}$ and define

$$M(c, y) = \sum_{t=1}^T (s_{c_t}(o_t) - s_{y_t}(o_t)),$$

$$M_\mu(c, y) = \sum_{t=1}^T (s_{c_t}(o_t + \mu_t) - s_{y_t}(o_t + \mu_t)).$$

Then the path-level margin shift is

$$\begin{aligned} \Delta M(c, y) &\triangleq M_\mu(c, y) - M(c, y) \\ &= \sum_{t=1}^T \Delta m_t(c_t) \\ &= \sum_{t=1}^T \langle w_{c_t} - w_{y_t}, \mu_t \rangle. \end{aligned}$$

If $\Delta M(c, y) < 0$, the perturbation reduces the margin of path c against the target path. The preceding identity shows that the cross-entropy term optimizes the same one-step shifts, but averaged over all competing tokens rather than selected along a fixed path. $\square$

Corollary G.5 (Value quantization induces an attention-concentration penalty). *Assume row-wise independent value noise $\Delta V = [\Delta v_1^\top; \dots; \Delta v_S^\top]$ with $\mathbb{E}[\Delta v_i] = 0$ and $\text{Cov}(\Delta v_i) = \sigma_V^2 I$. Then*

$$\mathbb{E} \|p \Delta V\|^2 = d_v \sigma_V^2 \|p\|_2^2.$$

Since $\|p\|_2^2$ is minimized by uniform attention and maximized by one-hot attention, value perturbations contribute a larger second-order penalty for overly concentrated attention.

Proof. $p \Delta V = \sum_i p_i \Delta v_i$. By independence and isotropic covariance, $\text{Cov}(p \Delta V) = \sum_i p_i^2 \sigma_V^2 I = \sigma_V^2 \|p\|_2^2 I$. Taking the trace yields the result. $\square$

Interpretation. The zero-mean special case explains the regularization effect: quantization adds a structured second-order penalty whose strength depends on the attention distribution and quantization scale. The learned non-uniform case can go further. By moving (R, T) with the downstream loss, the quantizer can introduce a small systematic perturbation μ that reduces high-probability competing margins. When this negative first-order effect dominates the positive curvature penalty in Proposition 3.2, the compressed KV cache can obtain lower local loss than the full-precision cache. This is the mathematical mechanism behind treating KV quantization as implicit regularization rather than only as reconstruction error, consistent with classic noise-as-regularization intuition (Bishop, 1995).

G.2 Hard Objective and Deployment Consistency

This subsection separates two facts that are often conflated in STE-based quantization: the hard empirical objective is not classically differentiable in thresholds, but the forward computation remains exactly the deployed quantizer.

Feasible set. Let $\varphi = (R, T)$, with $R \in \mathcal{R}$ for a compact convex set $\mathcal{R} \subset \mathbb{R}^K$ and

$$0 \leq t_1 < \dots < t_{K-1} \leq 1.$$

During optimization, this strict ordering is maintained by projection with a small margin $\delta_{\text{thr}} > 0$. For the projected-optimizer statement below, define the closed feasible set

$$\begin{aligned} \Phi = \{ (R, T) : R \in \mathcal{R}, \\ 0 \leq t_1, \quad t_k + \delta_{\text{thr}} \leq t_{k+1}, \\ 1 \leq k < K-1, \\ t_{K-1} \leq 1 \}. \end{aligned}$$

Hard empirical objective. For a finite calibration set $\{x_i\}_{i=1}^n$ and per-sample downstream losses L_i , define

$$F_n(\varphi) = \frac{1}{n} \sum_{i=1}^n L_i(Q(x_i; \varphi)).$$

Proposition G.6 (Piecewise-constant threshold objective). *Fix R and a finite set $\{x_i\}_{i=1}^n$. On any open region of threshold space where no threshold crosses any sample, i.e., $t_k \neq x_i$ for all i, k , the bin assignments are constant and F_n is constant as a function of T . Consequently, the classical derivative of F_n with respect to the thresholds is zero almost everywhere.*

Proof. Inside such a region, every indicator $\mathbb{I}\{x_i \in B_k(T)\}$ is fixed. Thus each $Q(x_i; R, T)$ is fixed, and so every term $L_i(Q(x_i; R, T))$ is fixed. The only possible changes occur on the finite collection of hyperplanes $t_k = x_i$, which has Lebesgue measure zero. $\square$

Proposition G.7 (No forward train-deploy gap). *For any parameter iterate $\varphi_t = (R_t, T_t)$ produced by hard-forward STE training, the forward value used during training equals the value used at inference:*

$$\tilde{x}_{\text{forward}} = Q(x; R_t, T_t).$$

Proof. From Eq. (6),

$$\tilde{x} = x + \text{sg}(Q(x; R_t, T_t) - x).$$

The stop-gradient operator is the identity in the forward pass, so $\tilde{x} = x + Q(x; R_t, T_t) - x = Q(x; R_t, T_t)$. It only changes the backward derivative. $\square$

Pseudo-gradient oracle. The hard-forward estimator is therefore best viewed as an inexact first-order oracle for quantizer parameters. The population update direction can be written abstractly as

$$g_{\text{PG}}(\varphi) = \mathbb{E} \left[J_{\text{PG}, \varphi}(x; \varphi)^\top \nabla_u L(u) \Big|_{u=Q(x; \varphi)} \right], \quad (9)$$

where $J_{\text{PG}, \varphi}$ is the pseudo-Jacobian induced by the input, level, and threshold update rules in Sec. 3.2. No smooth sigmoid, softmax relaxation, temperature, or annealing schedule is part of this oracle.

G.3 Optimizer Sanity Check for an Inexact STE Oracle

We update only $\varphi = (R, T)$ and freeze the backbone weights. The hard empirical objective in Appendix G.2 is piecewise constant in thresholds, so this subsection is not a convergence theory for the classical derivative of that hard objective. It is a conditional optimization sanity check: if the hard-forward backward rule is treated as a stochastic first-order oracle for a smooth reference objective f , with a uniformly bounded oracle mismatch, then the standard projected AMSGrad stationarity bound applies. This separates the deployable hard forward pass from the reference first-order geometry used only for optimization.

Gradient mapping. For a differentiable reference objective f on the closed convex set Φ , define

$$\mathcal{G}_\eta(\varphi) = \frac{1}{\eta} (\varphi - \Pi_\Phi(\varphi - \eta \nabla f(\varphi))),$$

where Π_Φ is Euclidean projection. If the projection is inactive, $\mathcal{G}_\eta(\varphi) = \nabla f(\varphi)$. Thus $\|\mathcal{G}_\eta(\varphi)\|$ is the constrained analogue of a gradient norm.

Projected AMSGrad update. Given a mini-batch hard-forward pseudo-gradient estimator g_t at

φ_t , projected AMSGrad uses

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (g_t \odot g_t), \\ \hat{v}_t &= \max\{\hat{v}_{t-1}, v_t\} \quad (\text{elementwise}), \\ \varphi_{t+1} &= \Pi_\Phi \left(\varphi_t - \eta_t \frac{m_t}{\sqrt{\hat{v}_t + \epsilon_{\text{adam}}}} \right), \end{aligned} \quad (10)$$

where $\odot$ is the Hadamard (elementwise) product, $\sqrt{\cdot}$ and all divisions are elementwise, and $\epsilon_{\text{adam}} > 0$ is the standard numerical-stability constant. Bias correction is omitted only to simplify notation; it changes constants through a deterministic rescaling of η_t . Adam uses the same m_t and v_t recursions (Kingma and Ba, 2015). AMSGrad replaces v_t by the monotone envelope $\hat{v}_t$, which is the ingredient used in the convergence proof (Reddi et al., 2018).

Assumption G.8 (Smooth reference objective and inexact STE oracle). Assume:

- (i) Φ is closed, convex, and bounded, and f is L_f -smooth on Φ , i.e., $\|\nabla f(x) - \nabla f(y)\| \leq L_f \|x - y\|$ for all $x, y \in \Phi$.
- (ii) f is bounded below on Φ by f^{inf} .
- (iii) With respect to the filtration $\mathcal{F}_t$ generated by the previous minibatches and iterates,

$$b_t \triangleq \mathbb{E}[g_t \mid \mathcal{F}_t] - \nabla f(\varphi_t)$$

is the oracle mismatch and is uniformly bounded by $\|b_t\| \leq \epsilon_{\text{PG}}$ almost surely.

- (iv) Stochastic pseudo-gradients are uniformly bounded in ℓ_∞ : $\|g_t\|_\infty \leq G$ almost surely for all t .
- (v) $\eta_t = \eta/\sqrt{t}$ for some $\eta > 0$, and $\beta_1, \beta_2 \in (0, 1)$ with $\beta_1^2 < \beta_2$.

Lemma G.9 (One-step descent with oracle mismatch). *Under Assumption G.8, there exist constants $a_0, a_1, a_2 > 0$, independent of t and T , such that the projected AMSGrad iterate satisfies*

$$\begin{aligned} \mathbb{E}_t[f(\varphi_{t+1})] &\leq f(\varphi_t) - a_0 \eta_t \|\mathcal{G}_{\eta_t}(\varphi_t)\|^2 \\ &\quad + a_1 \eta_t |\langle \mathcal{G}_{\eta_t}(\varphi_t), b_t \rangle| + a_2 \eta_t^2, \end{aligned}$$

where $\mathbb{E}_t[\cdot] = \mathbb{E}[\cdot \mid \mathcal{F}_t]$.

Proof. Let $\bar{g}_t = \mathbb{E}[g_t \mid \mathcal{F}_t] = \nabla f(\varphi_t) + b_t$ and write the martingale noise as $\xi_t = g_t - \bar{g}_t$, so

$\mathbb{E}_t[\xi_t] = 0$. The bounded-gradient assumption implies $\epsilon_{\text{adam}} \leq \sqrt{\hat{v}_{t,i}} + \epsilon_{\text{adam}} \leq G + \epsilon_{\text{adam}}$ for every coordinate i . Hence the diagonal AMSGrad preconditioner is uniformly positive definite and uniformly bounded; all norm changes between the preconditioned update and the Euclidean gradient mapping are controlled by constants depending only on $G, \epsilon_{\text{adam}}$, and the dimension of φ .

By L_f -smoothness,

$$f(\varphi_{t+1}) \leq f(\varphi_t) + \langle \nabla f(\varphi_t), \varphi_{t+1} - \varphi_t \rangle + \frac{L_f}{2} \|\varphi_{t+1} - \varphi_t\|^2.$$

The projection optimality condition for $\varphi_{t+1} = \Pi_{\Phi}(\varphi_t - \eta_t D_t^{-1} m_t)$, where $D_t = \text{Diag}(\sqrt{\hat{v}_t} + \epsilon_{\text{adam}})$, gives the usual projected-descent inequality: the component of the step aligned with the reference gradient contributes a negative term proportional to $\eta_t \|\mathcal{G}_{\eta_t}(\varphi_t)\|^2$. The replacement of $\nabla f(\varphi_t)$ by the conditional mean $\bar{g}_t = \nabla f(\varphi_t) + b_t$ leaves the additional inner product $\eta_t \langle \mathcal{G}_{\eta_t}(\varphi_t), b_t \rangle$. Finally, the martingale noise ξ_t , first-moment recursion, and bounded preconditioner contribute only second-order terms after conditional expectation; the AMSGrad monotonicity of $\hat{v}_t$ and the condition $\beta_1^2 < \beta_2$ control the accumulated momentum/preconditioner variation by a constant multiple of η_t^2 . Collecting these standard AMSGrad terms yields the stated inequality with constants a_0, a_1, a_2 independent of t and T . $\square$

Theorem G.10 (Projected AMSGrad stationarity for the reference objective). *Let f be the smooth reference objective in Assumption G.8. Then projected AMSGrad with $\eta_t = \eta/\sqrt{t}$ satisfies*

$$\begin{aligned} \min_{1 \leq t \leq T} \mathbb{E} \|\mathcal{G}_{\eta_t}(\varphi_t)\|_2^2 &\leq \frac{C_1 + C_2(1 + \log T)}{\sqrt{T}} + C_3 \epsilon_{\text{PG}}^2, \end{aligned}$$

for constants $C_1, C_2, C_3 > 0$ independent of T . Thus the projected gradient mapping of the reference objective decays at the standard nonconvex projected-AMSGrad rate up to the STE oracle mismatch.

Proof of Theorem G.10. Let $H_t = \mathcal{G}_{\eta_t}(\varphi_t)$. Taking total expectation in Lemma G.9, summing from $t = 1$ to T , and using $f(\varphi_{T+1}) \geq f^{\text{inf}}$ gives the following bound, where

$$\begin{aligned} A_0 &= a_0^{-1}(f(\varphi_1) - f^{\text{inf}}), \\ A_1 &= a_0^{-1}a_2, \quad A_2 = a_0^{-1}a_1. \end{aligned}$$

$$\begin{aligned} \sum_{t=1}^T \eta_t \mathbb{E} \|H_t\|^2 &\leq A_0 + A_1 \sum_{t=1}^T \eta_t^2 \\ &\quad + A_2 \sum_{t=1}^T \eta_t \mathbb{E} |\langle H_t, b_t \rangle|. \end{aligned} \tag{11}$$

The first term is the finite initial optimality gap, the second term is the usual stochastic/adaptive-stepsize accumulation, and the third term is the only term introduced by the hard-forward oracle mismatch. By Cauchy–Schwarz and Young’s inequality, for any $\rho > 0$,

$$\begin{aligned} |\langle H_t, b_t \rangle| &\leq \frac{\rho}{2} \|H_t\|^2 + \frac{1}{2\rho} \|b_t\|^2 \\ &\leq \frac{\rho}{2} \|H_t\|^2 + \frac{1}{2\rho} \epsilon_{\text{PG}}^2. \end{aligned}$$

Choosing $\rho = 1/A_2$ and substituting this bound into Eq. (11) gives one half of $\sum_t \eta_t \mathbb{E} \|H_t\|^2$ back on the right-hand side. Moving it to the left yields

$$\begin{aligned} \sum_{t=1}^T \eta_t \mathbb{E} \|H_t\|^2 &\leq 2A_0 + 2A_1 \sum_{t=1}^T \eta_t^2 \\ &\quad + A_2^2 \epsilon_{\text{PG}}^2 \sum_{t=1}^T \eta_t. \end{aligned} \tag{12}$$

Since all η_t are positive, the smallest stationarity measure among the first T iterates is bounded by the stepsize-weighted average:

$$\min_{1 \leq t \leq T} \mathbb{E} \|\mathcal{G}_{\eta_t}(\varphi_t)\|^2 \leq \frac{\sum_{t=1}^T \eta_t \mathbb{E} \|H_t\|^2}{\sum_{t=1}^T \eta_t}.$$

For $\eta_t = \eta/\sqrt{t}$, $\sum_{t=1}^T \eta_t \geq \eta\sqrt{T}$ and $\sum_{t=1}^T \eta_t^2 \leq \eta^2(1 + \log T)$. Combining these inequalities with Eq. (12) yields

$$\begin{aligned} \min_{1 \leq t \leq T} \mathbb{E} \|\mathcal{G}_{\eta_t}(\varphi_t)\|^2 &\leq \frac{2A_0 + 2A_1 \eta^2(1 + \log T)}{\eta\sqrt{T}} \\ &\quad + A_2^2 \epsilon_{\text{PG}}^2. \end{aligned}$$

Setting $C_1 \triangleq 2A_0/\eta$, $C_2 \triangleq 2A_1\eta$, and $C_3 \triangleq A_2^2$ gives the claimed bound. $\square$

G.4 Scope of the Mathematical Claims

The appendix makes three limited claims. First, **Beyond** has no forward train–deploy mismatch: training and inference evaluate the same hard quantizer. Second, because the hard empirical objective is piecewise constant in thresholds, threshold learning relies on an explicitly biased boundary pseudo-gradient. Third, the Adam/AMSGrad statement is

a first-order stationarity guarantee for a smooth reference objective under a bounded-oracle-mismatch STE update; it is not a global-optimality guarantee for the piecewise-constant hard objective. These are the assumptions under which the standard input STE and hard-forward threshold update in the main paper should be interpreted.